TECHNICAL ARCHITECTURE BRIEF

Meta-Analysis 101 — AI-Assisted Workshop Design

How the Midterm and Final workshops use AI to guide learners through a complete meta-analysis — while keeping all R code deterministic and error-free.

---

1. Platform overview

Meta-Analysis 101 is a bilingual (Traditional Chinese / English) interactive educational platform that teaches systematic review and meta-analysis methodology to pharmacists and clinical researchers. The hands-on workshop is split into two phases:

Phase	Component	Steps	AI calls	Focus
Phase 1	Midterm.jsx	5	2	Planning: PICO, search, studies, extraction, RoB
Phase 2	Final.jsx	5	Up to 4	Analysis: effect size, run (WebR), interpret, conclude

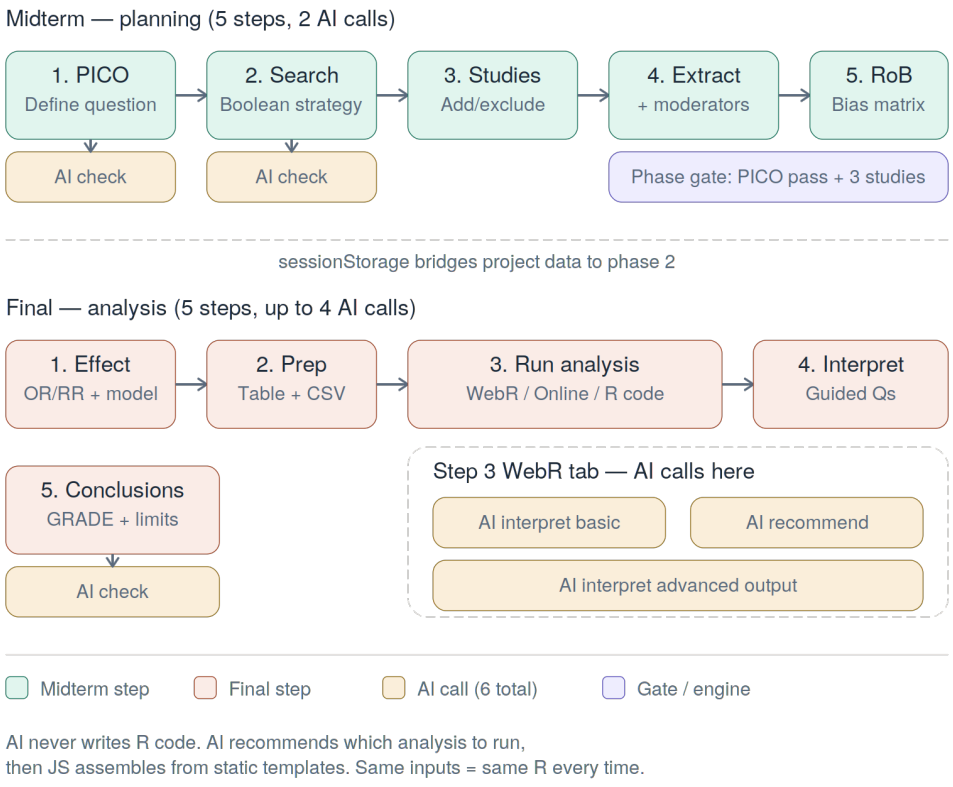

Figure 1 — Both workshop phases with all AI touchpoints and the session bridge between them.

2. Midterm workshop — planning phase

Step	What the user does	AI role

1. Define PICO	Enters clinical question + P/I/C/O fields	AI validates specificity, searchability, feasibility
2. Search strategy	Picks databases, writes Boolean query	AI checks syntax, coverage, MeSH terms
3. Add studies	Adds study cards with citation, design, N	None
4. Data extraction	Enters outcome data + moderator variables	None
5. Risk of bias	Fills traffic-light RoB matrix	None

Phase gate: PICO must pass AI check and at least 3 studies must be included before proceeding. **Moderator columns** (Step 4) let users tag studies with variables like region or dosage for subgroup analysis in Phase 2.

3. Final workshop — analysis phase

Step	What the user does	AI role
1. Effect + model	Chooses OR/RR/MD/SMD + fixed/random	None
2. Prepare data	Reviews table; downloads CSV	None
3. Run analysis	WebR in-browser / Online calc / R code	Up to 3 calls (Section 4)
4. Interpret	Answers guided questions	None
5. Conclusions	Finding, GRADE, limitations, implications	AI checks consistency

Step 3 is the technical core: the WebR tab runs real R (*metafor* package) in the browser via WebAssembly, producing genuine forest plots, funnel plots, and statistical output identical to desktop RStudio.

4. Step 3 WebR tab — the two-layer architecture

> ***The differentiator:*** *Other MA websites run static, pre-defined analyses. MA101 uses AI to judge which advanced analyses are appropriate for the user's specific dataset, then assembles and executes the corresponding R code — all in-browser. The experience feels conversational and custom; the code is safe and deterministic.*

Layer 1 — Basic analysis (no AI in code generation)

The user's choices feed into `buildRCode()`, a pure JavaScript function that deterministically assembles R code. Same inputs = same R string every time. WebR executes it, producing forest plot, funnel plot, and stats (pooled effect, CI, I^2, Q, tau^2). After results display, the user clicks **"AI Interpret Results"** — this is **AI Call 1**.

Layer 2 — Advanced analysis: AI's three distinct roles

This is where AI plays its most distinctive role — not just interpreting, but actively advising the researcher on what to explore next.

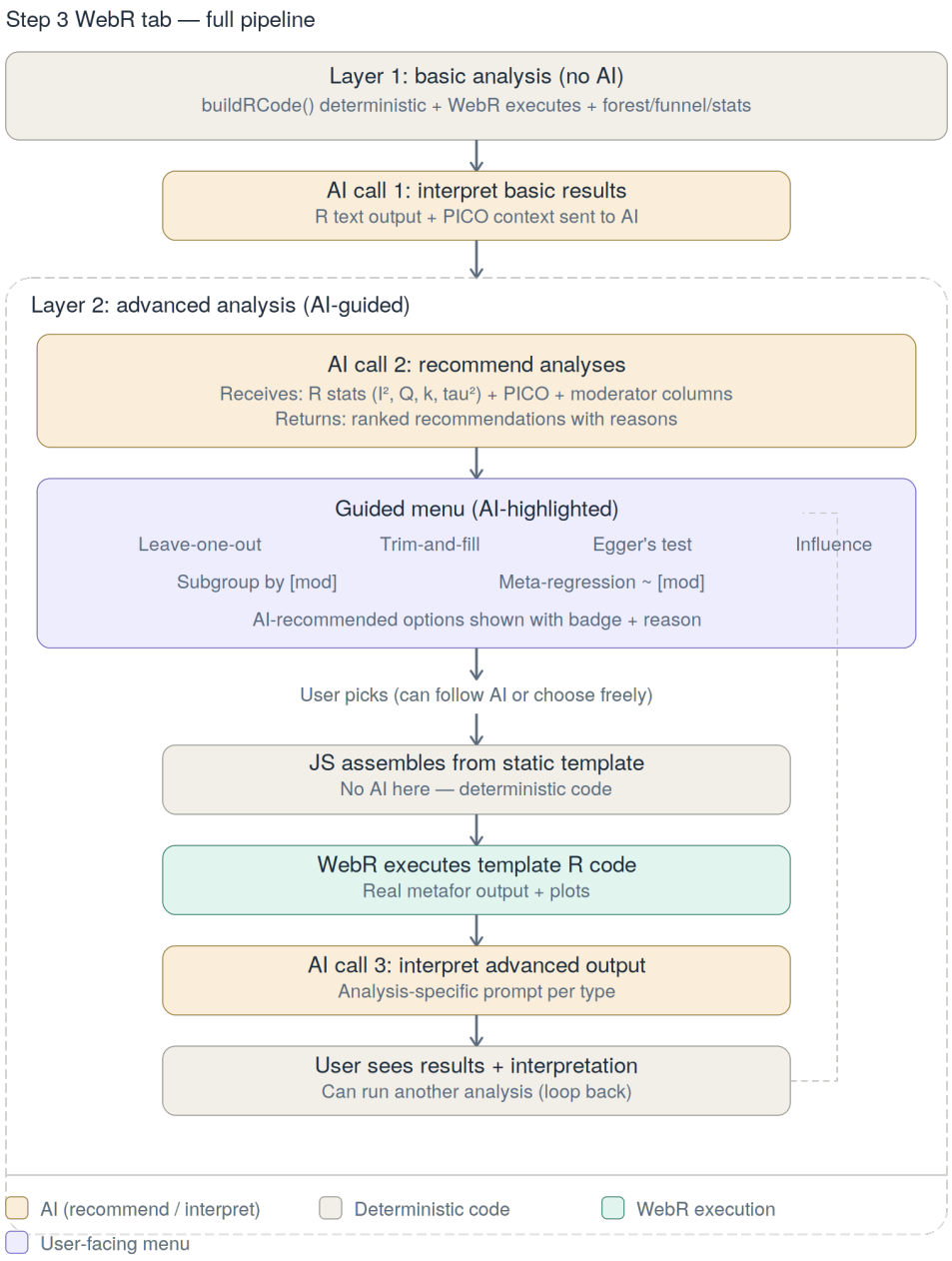

Figure 2 — The complete Layer 2 pipeline: AI recommends, user picks, JS assembles templates, WebR executes, AI interprets.

Role 1 — Recommend (AI Call 2): AI receives the Layer 1 R output (I^2, Q, k, tau^2) + PICO context + available moderator columns. It returns ranked recommendations with reasons, e.g.: "Your I^2 is 35% with 5 studies — I recommend leave-one-out to check robustness, and subgroup by Population Focus since your studies mix CKD-specific and high-CV-risk populations." These appear as highlighted options in the guided menu.

Role 2 — Never touch code: The user picks from the menu (AI-highlighted or freely chosen). JS validates against a whitelist and slots parameters into pre-written, pre-tested R template blocks. AI is not involved in code assembly.

Analysis type	R template	Needs moderator?
Leave-one-out	leave1out(res)	No
Trim-and-fill	trimfill(res); funnel(tf)	No

Egger's regression	regtest(res)	No
Influence diagnostics	influence(res); plot(inf)	No
Subgroup analysis	rma() per subgroup level	Yes (categorical)
Meta-regression	rma(mods=~moderator)	Yes

Role 3 — Interpret advanced output (AI Call 3): After WebR executes the template code, AI reads the new R output and provides analysis-specific interpretation. The user can then run another analysis, looping back to the menu.

5. The safety firewall — what AI does vs. does not touch

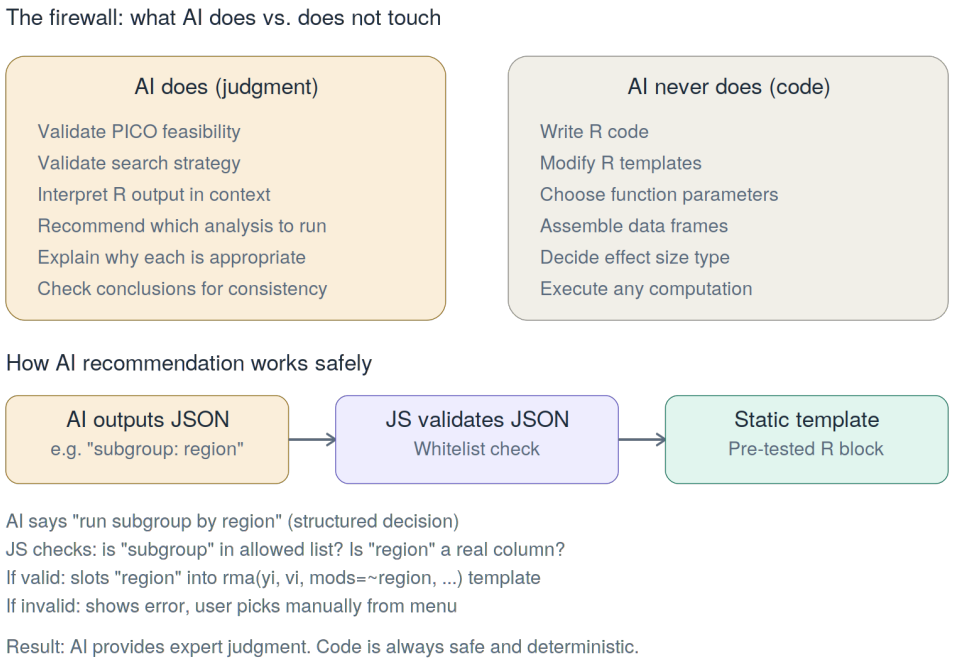

Figure 3 — The AI/code firewall: AI provides judgment, JS provides safe execution.

How the recommendation step stays safe: AI outputs a structured decision (e.g. "subgroup analysis by Region"). JavaScript validates: Is "subgroup" an allowed type? Is "Region" an actual column? If valid, JS slots it into the pre-tested template. If invalid, fallback to the manual menu. AI never sees or modifies any R code.

6. Complete AI call inventory

#	Checkpoint	Location	Trigger
1	PICO validation	Midterm Step 1	User clicks AI Check
2	Search strategy check	Midterm Step 2	User clicks AI Check
3	Interpret basic results	Final Step 3 (Layer 1)	User clicks AI Interpret
4	Recommend analyses	Final Step 3 (Layer 2)	Automatic after Layer 1
5	Interpret advanced output	Final Step 3 (Layer 2)	User runs advanced analysis

6	Check conclusions	Final Step 5	User clicks AI Check

Most users hit 4 calls per full workshop run. Estimated cost: ~$0.01-0.03 per run. All calls gated behind login + course completion.

7. Why this design matters

	Other MA websites	Meta-Analysis 101
Basic analysis	User inputs data, static code runs	Same — deterministic R via WebR
Advanced	User must know what to run	AI evaluates results, recommends what and why
Interpretation	User reads raw output alone	AI provides PICO-aware interpretation
Code safety	N/A (fixed scripts)	AI never writes code — JS uses templates
Teaching	Computation tool only	Guided methodology + computation + AI feedback

> ***In summary:*** *AI acts as a methodology advisor — it looks at the user's specific results and tells them what to explore next and why. All code generation remains deterministic. Same inputs = same R code = same results. AI provides expert judgment; JavaScript provides reliable execution.*

8. Technology stack

Layer	Technology	Role
Frontend	React (inline styles)	UI, wizard flow, state management
In-browser R	WebR (WASM) + metafor	Real R execution, forest/funnel plots
AI proxy	Vercel serverless + Anthropic API	AI feedback calls (key hidden server-side)
Backend	Supabase (PostgreSQL)	Auth, progress tracking, AI gating
Hosting	Vercel	Deployment, preview branches
Bilingual	Custom useI18n() hook	Traditional Chinese / English